



UNIVERSIDAD
DE GRANADA



Cátedra de Inteligencia Artificial

Universidad de Granada – Unicaja

Agente Conversacional Bancario

Integrantes

Pablo Martín Palomino `pablomarpa@correo.ugr.es`

Pablo Reyes Sousa `sousareyp@correo.ugr.es`

Natalia Yue Gallardo Pretel `nataliayue@correo.ugr.es`

Septiembre de 2026

Índice

1	Introducción y arquitectura general	2
1.1	Objetivos y enfoque de la solución	2
1.2	Arquitectura técnica	2
2	Interacción Natural y Experiencia de Usuario (UX)	3
2.1	Integración de voz y texto	3
2.2	Generación de respuestas en streaming	3
2.3	Interfaz adaptativa y accesible	3
3	Configuración, Generación y Seguridad de la Base de Datos	3
3.1	Generación de un histórico realista	4
3.2	Defensa en profundidad y aislamiento del LLM	4
4	Motor de Inteligencia Artificial	5
4.1	Gestión del historial y prevención del desbordamiento de contexto	5
4.2	System Prompt dinámico y reglas de comportamiento	5
4.3	Uso de Tool Calling unificado	6
5	Operaciones Bancarias Seguras	6
5.1	Atajos de latencia	6
5.2	Flujo transaccional de Bizum	7
6	Consultas y Analítica Avanzada	8
6.1	Traducción del lenguaje natural a SQL para el histórico de movimientos	8
6.2	Enfoque híbrido: heurística y estadística	8
6.2.1	Detección de pagos recurrentes y suscripciones	8
6.2.2	Proyecciones de gasto y analítica predictiva	9
7	Motor Visual Generativo	9
7.1	Generative UI y separación de responsabilidades	9
7.2	Gráficos Vega-Lite y tablas dinámicas sin plantillas	10
8	Conclusiones y Cumplimiento del Baremo	11
8.1	Conclusiones	11
8.2	Cumplimiento del baremo técnico del reto	11

1. Introducción y arquitectura general

1.1. Objetivos y enfoque de la solución

El proyecto “Habla con tu dinero” es un asistente conversacional desarrollado como respuesta al reto propuesto por la Cátedra de Inteligencia Artificial de la Universidad de Granada y Unicaja. Su objetivo principal es ofrecer al usuario una forma sencilla e interactiva de consultar y gestionar información relacionada con sus finanzas. Para ello, permite realizar consultas sobre cuentas, movimientos y productos financieros, además de ejecutar determinadas operaciones.

Para garantizar un funcionamiento fiable, se ha separado la interpretación de las consultas de la ejecución de las operaciones. El modelo de lenguaje se utiliza para interpretar las peticiones realizadas en lenguaje natural, identificar la intención del usuario y determinar qué información es necesaria para responder. Los cálculos, las consultas sobre los datos y las operaciones financieras se realizan directamente mediante la lógica de la aplicación.

Esta separación permite controlar mejor el funcionamiento del sistema. Mientras que el modelo de lenguaje facilita la interacción con el usuario, el backend se encarga de obtener los datos, realizar los cálculos y ejecutar las operaciones correspondientes. De esta manera, las respuestas relacionadas con información financiera se basan siempre en los datos disponibles en la aplicación.

1.2. Arquitectura técnica

La aplicación se divide en tres bloques principales: la interfaz de usuario, el backend y la base de datos.

- **Interfaz de usuario (Frontend):** Se ha desarrollado utilizando HTML y JavaScript, sin recurrir a frameworks complejos. La comunicación con el servidor se realiza mediante WebSockets, manteniendo una conexión abierta durante la interacción. Esto permite recibir las respuestas de forma progresiva y actualizar la interfaz dinámicamente. Además, se pueden mostrar elementos como gráficos, tablas y mensajes dentro de la propia conversación.
- **Motor principal (Backend):** Se ha desarrollado en Python utilizando FastAPI y constituye el núcleo de la aplicación. Se encarga de recibir las peticiones del usuario, gestionar el contexto de la conversación y coordinar los diferentes componentes del sistema. También permite resolver directamente determinadas consultas sin utilizar el modelo de lenguaje. Por ejemplo, una consulta sobre el saldo de una cuenta puede resolverse directamente desde el backend, reduciendo el tiempo de respuesta.
- **Base de datos y seguridad:** La información de la aplicación se almacena en una base de datos SQLite. El acceso a los datos se gestiona desde el backend, que determina qué operaciones puede realizar cada parte del sistema. Las consultas que únicamente necesitan información utilizan acceso de lectura, mientras que las operaciones que modifican datos o implican movimientos de dinero son ejecutadas directamente por la lógica de la aplicación.

2. Interacción Natural y Experiencia de Usuario (UX)

2.1. Integración de voz y texto

2.2. Generación de respuestas en streaming

Las respuestas del asistente se transmiten de forma progresiva mediante una conexión WebSocket entre el *frontend* y el *backend*. De esta forma, el usuario no tiene que esperar a que se genere la respuesta completa para empezar a visualizarla, sino que recibe el contenido a medida que está disponible.

El uso de WebSockets permite mantener una comunicación continua entre el cliente y el servidor y enviar distintos tipos de información durante una misma interacción. Además del texto de la respuesta, por este canal pueden transmitirse actualizaciones relacionadas con los datos mostrados en pantalla, como cambios en el saldo o instrucciones para representar tablas y gráficos.

Esta estrategia reduce el tiempo de espera percibido y permite que la interfaz comience a mostrar información mientras el procesamiento de la consulta continúa en segundo plano. De esta forma, la generación de la respuesta y la actualización de los distintos elementos de la interfaz pueden realizarse de forma progresiva.

2.3. Interfaz adaptativa y accesible

La interfaz se adapta al tipo de información que se obtiene en cada consulta. Además de las respuestas en texto, el sistema puede mostrar tablas y gráficos cuando estos aportan información adicional al usuario. Por ejemplo, una consulta sobre la evolución de los gastos puede complementarse con una representación gráfica de los datos obtenidos.

La generación de estos elementos visuales se ha separado del proceso principal de generación de la respuesta para evitar que afecte innecesariamente al tiempo de respuesta. Para controlar esta decisión se utiliza la variable `VISUALES_AL_LLM`, que permite configurar si la decisión de utilizar una herramienta visual se delega en el modelo de lenguaje o se realiza directamente desde el backend.

Durante las pruebas se observó que, cuando el modelo de lenguaje podía decidir libremente cuándo utilizar las herramientas de generación de gráficos, la respuesta podía verse interrumpida para realizar dicha operación. Esto incrementaba la latencia hasta alcanzar aproximadamente 12,4 segundos.

Para evitar este comportamiento, en la configuración actual las herramientas visuales no se exponen al modelo principal. En su lugar, el backend determina de forma determinista, a partir de los datos obtenidos, si es necesario generar una tabla o un gráfico. Cuando corresponde, esta generación se ejecuta de forma paralela a la respuesta de texto, de manera que ambas tareas no tengan que esperarse entre sí.

Con este cambio, el tiempo de respuesta se redujo hasta aproximadamente 6,7 segundos. Además de mejorar la latencia, esta separación permite mantener independientes la generación de la respuesta y la representación visual de los datos.

3. Configuración, Generación y Seguridad de la Base de Datos

3.1. Generación de un histórico realista

Para realizar las pruebas del asistente es necesario disponer de un conjunto de datos que permita representar diferentes situaciones financieras. Para ello, el proyecto incluye un script de generación sintética, `seed.py`, que crea un histórico de operaciones correspondiente a un periodo de 24 meses.

El generador utiliza una semilla determinista mediante `random.seed(42)`, lo que permite obtener siempre los mismos datos a partir de una misma configuración. Esto facilita tanto el desarrollo como la repetición de las pruebas, ya que los resultados se pueden comparar sobre un conjunto de datos conocido.

Los datos generados no se limitan a operaciones aleatorias, sino que incluyen diferentes tipos de movimientos y patrones que permiten probar las consultas del asistente. Entre ellos se encuentran:

- **Gastos periódicos y variables:** se generan recibos con distintas periodicidades, como pagos mensuales o bimestrales, junto con gastos variables relacionados con categorías como supermercado o combustible.
- **Pagos con periodicidad anual:** se incluyen operaciones separadas por periodos de tiempo más amplios, como la renovación anual del seguro del coche. Esto permite comprobar el funcionamiento de las consultas que requieren aplicar filtros sobre periodos de tiempo extensos.
- **Cambios en los gastos:** se introducen modificaciones en determinadas operaciones a lo largo del histórico. Por ejemplo, una suscripción puede aumentar su precio a mitad del periodo, permitiendo comprobar si el sistema es capaz de identificar este tipo de variaciones.

De esta forma, la base de datos proporciona un conjunto de pruebas más variado que permite comprobar tanto consultas sencillas como aquellas que requieren analizar información de diferentes periodos.

3.2. Defensa en profundidad y aislamiento del LLM

Una parte importante del sistema es controlar el acceso del modelo de lenguaje a la base de datos. El modelo puede generar consultas para obtener información a partir de las peticiones realizadas por el usuario, por lo que es necesario evitar que dichas consultas puedan modificar los datos almacenados.

Para ello, el acceso a la base de datos, implementado en `database.py`, incorpora varias medidas de protección independientes:

- **Acceso de solo lectura:** cuando el modelo necesita consultar información del historial, el backend utiliza una conexión SQLite en modo solo lectura mediante `mode=ro`. De esta forma, las operaciones que intenten modificar la base de datos, como `DROP` o `UPDATE`, son rechazadas por el propio motor de SQLite.
- **Validación de las consultas:** antes de ejecutar una consulta generada por el modelo, el sistema comprueba su contenido mediante una serie de expresiones regulares. Se bloquean palabras clave asociadas a operaciones de modificación de datos o de la estructura de la base de datos, como `INSERT`, `DELETE` o `ALTER`. Además, se comprueba que la consulta comience por `SELECT` o `WITH`.

- **Limitación del número de resultados:** las consultas que devuelven un número elevado de registros pueden generar una cantidad de información innecesaria para el modelo. Para evitarlo, el sistema establece un límite de 50 filas mediante la constante `MAX_FILAS = 50`. Cuando una consulta devuelve más resultados, estos se truncan antes de incorporarse al contexto de la conversación.

Estas medidas permiten separar la generación de las consultas de su ejecución. El modelo de lenguaje se encarga de construir la consulta a partir de la petición del usuario, mientras que el backend controla qué consultas pueden ejecutarse y con qué permisos se accede a la base de datos.

4. Motor de Inteligencia Artificial

4.1. Gestión del historial y prevención del desbordamiento de contexto

El modelo de lenguaje trabaja con una ventana de contexto limitada. En el caso de los modelos utilizados en el proyecto, esta ventana puede ser de hasta 8192 tokens. Parte de este espacio se utiliza para las instrucciones del sistema y la definición de las herramientas disponibles. Por tanto, si el historial de una conversación crece demasiado, es necesario limitar la cantidad de mensajes que se mantienen en el contexto enviado al modelo.

Para gestionar esta situación, la clase `Agente`, definida en `agent.py`, incorpora la función `recortar_historial`. Esta función limita el historial a los últimos 24 mensajes, manteniendo siempre el primer mensaje de la conversación, que contiene el *System Prompt*.

La eliminación de mensajes tampoco se realiza de forma arbitraria. El algoritmo busca el inicio de un turno del usuario (`role: user`) antes de recortar el historial. De esta forma, se evita eliminar únicamente una parte de una interacción que incluya una llamada a una herramienta y su correspondiente respuesta. Mantener estas partes de la conversación en el orden correcto es necesario para que los mensajes enviados posteriormente al modelo mantengan una estructura válida.

De esta manera, el historial se mantiene dentro de un tamaño controlado sin eliminar las instrucciones principales del sistema ni romper la estructura de las interacciones realizadas mediante herramientas.

4.2. System Prompt dinámico y reglas de comportamiento

El comportamiento del asistente se configura mediante un *System Prompt* definido en `config.py`. En él se establecen las instrucciones que debe seguir el modelo, así como la información necesaria para interpretar las peticiones y utilizar las herramientas disponibles.

El *prompt* se genera dinámicamente durante el arranque de la aplicación. Entre la información que se incorpora se encuentra la fecha actual, obtenida mediante `date.today().isoformat()`. Esto permite proporcionar al modelo una referencia temporal concreta para interpretar consultas que utilizan expresiones como “este mes” o “la semana pasada”.

También se incorpora el esquema de la base de datos mediante `ESQUEMA_BD`. De esta forma, el modelo dispone de información sobre las tablas y columnas disponibles cuando necesita generar una consulta sobre los datos almacenados.

Además, el *System Prompt* establece determinadas reglas para mantener un formato de respuesta consistente. Entre ellas se encuentran las indicaciones relacionadas con el formato de las cantidades monetarias, el tratamiento del usuario de forma directa y las restricciones sobre el uso de Markdown.

4.3. Uso de Tool Calling unificado

El asistente utiliza *Tool Calling* para solicitar al backend la ejecución de determinadas operaciones. Las herramientas disponibles se definen en `tools.py` mediante esquemas que especifican los argumentos que puede utilizar cada una. Entre estas operaciones se encuentran las consultas de información, el envío de Bizums y la proyección de gastos.

Para evitar depender de un formato concreto de herramientas, el sistema incluye la función `adaptar_herramientas_a_openai`, que adapta las definiciones internas al formato utilizado por las APIs compatibles con el esquema de OpenAI. Esto permite utilizar diferentes proveedores o modelos manteniendo la misma estructura de herramientas en el backend.

El funcionamiento de una interacción con herramientas sigue varios pasos. En primer lugar, el modelo recibe la petición del usuario y determina si necesita utilizar alguna de las herramientas disponibles. Si es necesario, genera un *tool call* con los argumentos correspondientes en formato JSON.

El backend recibe esta petición y ejecuta la herramienta de forma local. Según el tipo de operación, esto puede implicar consultar la base de datos, realizar un cálculo o ejecutar una operación definida por la aplicación. Una vez obtenido el resultado, el backend lo incorpora a la conversación mediante un mensaje con el rol `tool`.

Finalmente, el modelo recibe el resultado de la herramienta junto con el resto del contexto y genera la respuesta que se muestra al usuario. De esta forma, el modelo se encarga de interpretar la petición y decidir qué herramienta utilizar, mientras que la ejecución de las operaciones se realiza en el backend.

5. Operaciones Bancarias Seguras

5.1. Atajos de latencia

Durante las pruebas se observó que algunas consultas frecuentes no requieren utilizar el modelo de lenguaje. Un ejemplo es la consulta sobre el saldo actual, que puede resolverse directamente desde el backend. Enviar este tipo de petición al LLM añadía aproximadamente 3 segundos al tiempo de respuesta sin aportar ninguna ventaja en este caso.

Para evitar este procesamiento innecesario, el backend, en `agent.py`, incorpora la función `es_consulta_saldo`, que permite identificar las consultas que solicitan únicamente el saldo actual. Cuando se cumple esta condición, la petición se resuelve directamente mediante la herramienta `consultar_saldo`, sin pasar por el modelo de lenguaje.

El filtro utiliza listas cerradas de palabras permitidas mediante `_RELLENO_SALDO`. Se ha optado por un criterio restrictivo para evitar que consultas que requieran información adicional sean identificadas incorrectamente como una consulta de saldo.

En este caso, los errores tienen distinto coste:

- **Falso negativo:** si una consulta sencilla no es identificada por el filtro, se envía al modelo de lenguaje. El sistema puede utilizar igualmente la herramienta `consultar_saldo`, aunque la respuesta tendrá una latencia mayor.
- **Falso positivo:** si una consulta más compleja se identifica incorrectamente como una consulta de saldo, el sistema podría devolver únicamente el saldo actual. Por ejemplo, una pregunta como “¿cuál era mi saldo el mes pasado?” requiere consultar información histórica y no debe resolverse mediante este atajo.

Por este motivo, el filtro prioriza evitar los falsos positivos, de forma que las consultas que puedan requerir un tratamiento adicional continúen su flujo normal hacia el modelo de lenguaje.

5.2. Flujo transaccional de Bizum

Las operaciones de envío de dinero se gestionan directamente desde el backend, sin delegar su ejecución en el modelo de lenguaje. El flujo se divide en varias etapas de validación y ejecución:

1. **Extracción y validación inicial:** la función `extraer_peticion_bizum` identifica en la petición del usuario los datos necesarios para realizar el envío, como el destinatario, el importe y el concepto. La extracción permite reconocer diferentes formas de expresar una misma operación. Antes de continuar con el proceso, el importe se comprueba para verificar que se encuentra dentro de los límites establecidos, entre 0,50€ y 1000€.
2. **Resolución del contacto:** una vez obtenido el destinatario, la función `buscar_contacto_bizum` comprueba los contactos disponibles. En caso de que el nombre proporcionado no coincida exactamente, se utilizan coincidencias parciales mediante `get_close_matches` para identificar posibles contactos. Si existe una coincidencia, esta se muestra al usuario antes de continuar con la operación.
3. **Autenticación desde el frontend:** cuando los datos de la operación son válidos, el estado de la conversación pasa a “Bizum pendiente” y el backend envía mediante WebSocket la señal `pedir_pin`. El frontend muestra entonces un teclado numérico para introducir el PIN. De esta forma, el PIN se introduce fuera del campo de texto de la conversación y no forma parte del historial que recibe el modelo de lenguaje.
4. **Validación y ejecución de la operación:** una vez validado el PIN, el backend ejecuta la función `api_enviar_bizum`. La modificación del saldo se realiza dentro de una transacción SQLite iniciada mediante `BEGIN IMMEDIATE`, utilizando `transaccion_escritura`. Esta instrucción establece el bloqueo necesario para evitar que dos operaciones de escritura se ejecuten simultáneamente sobre los mismos datos. De esta forma, la comprobación del saldo y su posterior actualización se realizan dentro de la misma transacción.

Este diseño mantiene separadas la interpretación de la petición y la ejecución de la operación bancaria. El modelo de lenguaje puede identificar la intención del usuario y proporcionar los datos necesarios, pero las validaciones y la modificación de los datos se realizan mediante la lógica del backend.

6. Consultas y Analítica Avanzada

6.1. Traducción del lenguaje natural a SQL para el histórico de movimientos

Una de las funcionalidades principales del asistente es permitir al usuario consultar su historial financiero utilizando lenguaje natural. Para ello, el sistema utiliza la técnica de *Text-to-SQL* mediante la herramienta `consultar_movimientos`. El modelo de lenguaje recibe la petición del usuario y genera una consulta en dialecto SQLite que permite obtener la información solicitada.

Para facilitar la generación de estas consultas, el *System Prompt* incluye una serie de reglas relacionadas con la estructura y el tratamiento de los datos:

- **Tratamiento de signos:** los gastos se almacenan con valores negativos y los ingresos con valores positivos. Por este motivo, cuando se necesita calcular el total de gastos, la consulta debe aplicar `SUM(-importe)` para obtener el resultado como una cantidad positiva en la respuesta.
- **Gestión temporal:** se proporcionan al modelo diferentes patrones para realizar filtros sobre fechas. Esto permite distinguir entre expresiones como “este mes” y “los últimos N meses”, utilizando las funciones de fecha disponibles en SQLite, como `strftime`.
- **Corrección de consultas:** si una consulta generada por el modelo produce un error durante su ejecución, la capa de acceso a la base de datos devuelve la información del error al agente. El modelo puede utilizar este resultado para corregir la sentencia SQL y volver a intentarlo. El número de reintentos está limitado para evitar ciclos indefinidos.

De esta forma, el modelo se utiliza para traducir la petición del usuario a una consulta sobre los datos, mientras que la ejecución de la sentencia y la gestión de los errores corresponden al backend.

6.2. Enfoque híbrido: heurística y estadística

Las consultas analíticas requieren operaciones que no resulta conveniente delegar directamente en el modelo de lenguaje. Por ejemplo, la identificación de pagos recurrentes o el cálculo de una proyección de gasto implican realizar cálculos sobre un conjunto de operaciones y detectar patrones en los datos.

Por este motivo, estas funcionalidades siguen un enfoque híbrido. El módulo `analitica.py` realiza los cálculos y obtiene los resultados a partir de los datos almacenados, mientras que el modelo de lenguaje se utiliza principalmente para interpretar la petición y redactar posteriormente la respuesta.

6.2.1. Detección de pagos recurrentes y suscripciones

La periodicidad de un pago no se almacena explícitamente en la base de datos, por lo que debe inferirse a partir del histórico de operaciones. El módulo `analitica.py` utiliza para ello la separación temporal entre los distintos cobros de un mismo concepto.

En lugar de utilizar únicamente el importe como criterio, el algoritmo analiza la regularidad de los intervalos entre pagos. Esto permite identificar pagos recurrentes aunque su importe varíe entre periodos, como puede ocurrir con recibos de suministros.

El procedimiento utiliza la mediana de los días transcurridos entre pagos y calcula su desviación típica. A partir de estos valores se obtiene un factor de irregularidad. Cuando este factor es menor o igual que 0,06 y la mediana se encuentra dentro de una cadencia conocida, como mensual, trimestral o anual, se considera que existe un patrón recurrente. Para la comparación con estas cadencias se utiliza un margen de tolerancia del 20 %.

Este procedimiento permite identificar diferentes tipos de pagos periódicos, como suscripciones o seguros anuales. Además, al disponer del histórico completo, el sistema puede detectar variaciones en el importe de un mismo pago a lo largo del tiempo.

6.2.2. Proyecciones de gasto y analítica predictiva

Para estimar el gasto total de un mes no resulta suficiente con extrapolar directamente el gasto acumulado hasta el momento. Esta aproximación puede producir resultados poco representativos al comienzo del mes, especialmente cuando existen pagos fijos que todavía no se han realizado.

El sistema divide la proyección en dos componentes:

- **Gastos fijos:** se identifican a partir de los pagos recurrentes detectados previamente. Su importe se incorpora a la proyección independientemente de que el pago se haya realizado ya durante el mes o esté pendiente.
- **Gastos variables:** se estiman a partir del ritmo de gasto del mes actual y de la media histórica de los últimos 12 meses. El peso de cada componente se modifica en función del número de días transcurridos del mes.

Durante el desarrollo se utilizó *backtesting* para comprobar el comportamiento de la proyección sobre datos históricos. Este proceso permite comparar las estimaciones que habría realizado el sistema con los gastos que realmente se produjeron y obtener una medida del error.

Los resultados del análisis se resumen antes de enviarlos al modelo de lenguaje. En lugar de proporcionarle únicamente los valores numéricos calculados, el backend puede generar conclusiones como “fiabilidad alta” o “tendencia por encima de la media”. De esta forma, el modelo recibe información ya procesada y se encarga principalmente de presentarla al usuario.

7. Motor Visual Generativo

7.1. Generative UI y separación de responsabilidades

El asistente puede complementar las respuestas de texto con componentes visuales generados a partir de los datos obtenidos en las consultas. Para ello, se utiliza un enfoque de *Generative UI*, en el que la estructura del componente visual se determina en función de la información disponible.

La generación de una visualización completa por parte del modelo de lenguaje, incluyendo tanto su estructura como los datos, presenta algunos inconvenientes. Por un lado, aumenta la cantidad de información que debe procesar el modelo y, por tanto, el tiempo

necesario para generar la respuesta. Por otro, permitir que el modelo reproduzca directamente los valores obtenidos de la base de datos puede introducir errores en los datos mostrados.

Para evitar estos problemas, el motor visual, implementado en `graficos.py`, separa la definición de la visualización de los datos que se muestran:

- **Decisión de viabilidad:** después de ejecutar una consulta SQL, el backend analiza el resultado para determinar si puede representarse mediante un componente visual. Por ejemplo, se puede considerar representable un resultado con varias filas y columnas numéricas, o una fila que contenga varias métricas que puedan compararse.
- **Diseño de la visualización:** cuando el resultado es apto para su representación, se realiza una llamada independiente al modelo de lenguaje. En esta llamada se proporcionan los nombres de las columnas y una muestra reducida de los datos. El modelo utiliza esta información junto con el contexto de la consulta para determinar si resulta más apropiado utilizar una tabla o un gráfico y generar la estructura correspondiente. La respuesta incluye también el campo `razonamiento`, que indica el motivo de la elección.
- **Inyección de los datos:** el modelo no genera los valores que se muestran finalmente al usuario. Devuelve únicamente la estructura del componente visual. El backend incorpora posteriormente los datos reales obtenidos de la base de datos antes de enviar el componente al frontend mediante WebSocket.

Esta separación permite que el modelo se encargue de determinar cómo representar la información, mientras que los datos mostrados proceden directamente de la consulta realizada por el backend.

7.2. Gráficos Vega-Lite y tablas dinámicas sin plantillas

El sistema no utiliza una colección fija de plantillas para representar los resultados. En su lugar, el modelo de lenguaje genera la estructura del componente a partir del tipo de información obtenida y de la consulta realizada.

- **Gráficos con Vega-Lite v5:** cuando los datos se pueden representar de forma más adecuada mediante una visualización, el modelo genera una especificación JSON compatible con Vega-Lite. En ella determina el tipo de marca utilizado mediante `mark`, como líneas, barras o áreas, y establece la correspondencia entre las columnas y los diferentes elementos del gráfico mediante `encoding`. También puede definir los títulos y etiquetas que se muestran en la interfaz.
- **Tablas dinámicas:** cuando resulta más importante consultar valores concretos o cuando los resultados contienen varios atributos, el modelo puede seleccionar una representación mediante tabla. En este caso, determina qué columnas deben mostrarse y en qué orden, además de asignar reglas de formato mediante etiquetas como “euros”, “fecha” o “texto”. El frontend utiliza esta información para representar cada columna con el formato correspondiente.

De esta forma, la representación visual se genera a partir de los resultados concretos de cada consulta, sin necesidad de definir previamente una plantilla específica para cada tipo de pregunta.

8. Conclusiones y Cumplimiento del Baremo

8.1. Conclusiones

El desarrollo de “Habla con tu dinero” ha permitido implementar un asistente conversacional capaz de consultar información financiera, realizar determinadas operaciones y generar representaciones visuales a partir de las peticiones del usuario.

Uno de los principales aspectos de la arquitectura es la separación de responsabilidades entre el modelo de lenguaje y el código de la aplicación. El LLM se utiliza principalmente para interpretar las peticiones, generar consultas SQL, seleccionar herramientas y definir la estructura de las visualizaciones. Por otro lado, el backend se encarga de realizar los cálculos, consultar los datos y ejecutar las operaciones que modifican el estado de la aplicación.

Esta separación también se aplica a las operaciones que requieren un mayor control, como el envío de Bizums. La validación de los datos, la autenticación mediante PIN y la actualización del saldo se realizan fuera del modelo de lenguaje y mediante la lógica del backend y las transacciones de la base de datos.

Durante el desarrollo también se han realizado diferentes ajustes para reducir el tiempo de respuesta. Un ejemplo es la resolución directa de determinadas consultas, como la consulta del saldo, y la generación paralela de los componentes visuales. En las pruebas realizadas, esta última modificación permitió reducir el tiempo de respuesta aproximado de 12,4 segundos a 6,7 segundos.

En conjunto, el proyecto muestra que es posible utilizar un modelo de lenguaje como parte de una aplicación de gestión financiera manteniendo fuera del modelo las operaciones que requieren cálculos, validaciones y modificaciones de los datos. Esta organización permite aprovechar el procesamiento de lenguaje natural sin depender del modelo para las partes críticas de la aplicación.

8.2. Cumplimiento del baremo técnico del reto

El desarrollo se ha realizado tomando como referencia los criterios de evaluación establecidos para el reto. Las principales funcionalidades implementadas son las siguientes:

- **Text-to-SQL (30 puntos):** se ha implementado la traducción de consultas en lenguaje natural a sentencias SQL mediante la herramienta `consultar_movimientos`. El esquema de la base de datos se incorpora al *System Prompt* y el sistema permite corregir determinadas consultas cuando se produce un error durante su ejecución. Las consultas sobre el histórico se realizan mediante una conexión SQLite de solo lectura (`mode=ro`) y se aplican validaciones adicionales antes de ejecutarlas.
- **Operaciones Bancarias (10 puntos):** el asistente permite realizar operaciones como la consulta del saldo y el envío de dinero mediante Bizum. El flujo de Bizum incluye la extracción y validación del importe, la resolución del destinatario, la introducción del PIN desde el frontend y la ejecución de la operación mediante una transacción SQLite iniciada con `BEGIN IMMEDIATE`.
- **Gráficos sin plantillas (10 puntos):** las visualizaciones se generan a partir de los resultados concretos de cada consulta, sin utilizar una plantilla fija para cada caso. El modelo genera especificaciones compatibles con Vega-Lite v5 y determina elementos como el tipo de gráfico, los ejes y los títulos.

- **Lógica visual (10 puntos):** el backend analiza inicialmente si los resultados obtenidos son adecuados para una representación visual. Cuando lo son, el modelo determina si la información debe mostrarse como tabla o como gráfico y genera la estructura correspondiente. La decisión incluye el campo **razonamiento**, utilizado para indicar el motivo de la elección.
- **Analítica avanzada (extra):** se han implementado funcionalidades adicionales para analizar los movimientos del usuario, como la detección de pagos recurrentes y la proyección del gasto mensual. Estos cálculos se realizan en Python mediante reglas y métodos estadísticos, mientras que el modelo de lenguaje se utiliza para interpretar la consulta y presentar los resultados.